HOSPITAL MANAGEMENT SYSTEM

BRANCH COVERAGE REPORT

Software Engineering Project · LLVM source-based coverage · 6 November 2026

PRIMARY RESULT	Branch coverage: 81.98% | 1,037 of 1,265 production branches

81.98%	228	86.67%	97.92%
Branch coverage	Uncovered branches	Line coverage	Function coverage

1. Purpose and scope

Branch coverage measures whether alternative outcomes of instrumented decision points were executed during the recorded test run. It complements line coverage: a line may execute while one or more decision outcomes on that line remain untested. This report evaluates the production build only.

2. Aggregate branch and supporting coverage

Metric	Covered	Missed	Total	Coverage
Lines	2,691	414	3,105	86.67%
Regions	1,638	178	1,816	90.20%
Functions	188	4	192	97.92%
Branches	1,037	228	1,265	81.98%

3. Branch coverage by production source file

Production source	Line coverage	Branch coverage
Appointment.cpp	83.68%	88.69%
Doctor.cpp	97.24%	95.45%
FileManager.cpp	77.65%	74.13%
HospitalSystem.cpp	90.31%	80.59%
Patient.cpp	97.80%	97.06%
main.cpp	100.00%	N/A

4. Detailed interpretation

Overall result. 1,037 of 1,265 instrumented branches were covered (81.98%); 228 branches (18.02%) were not observed in the recorded run. These uncovered outcomes warrant risk-based review, especially where they represent validation failures, persistence errors, or security checks.

FileManager.cpp — 74.13%. This is the lowest branch result. Prioritize alternate outcomes in CSV parsing/escaping, file open and write failures, malformed records, identity checks, and approved data-path validation. Security-related checks must remain enabled in production builds.

HospitalSystem.cpp — 80.59%. Review conditional paths in patient/doctor workflows, appointment booking and cancellation, search/filter operations, and validation/error feedback. Add tests for both accepted and rejected inputs where the requirements define both outcomes.

Appointment.cpp — 88.69%. Review uncovered branches in appointment state transitions, date/time validation, and conflict-related decisions. Existing appointment tests cover important workflows, but coverage does not establish that every boundary condition has been exercised.

Doctor.cpp and Patient.cpp — 95.45% and 97.06%. These components have comparatively strong branch coverage. Remaining uncovered outcomes should still be checked against validation and error-handling requirements before considering them low risk.

5. Recommended branch-focused test additions

Priority	Target	Suggested test focus
High	FileManager.cpp	Invalid/missing paths, symlink rejection, CSV parse/escape edge cases, read/write failures
High	HospitalSystem.cpp	Invalid workflow inputs, unavailable entities, rejected appointment transitions
Medium	Appointment.cpp	Date/time boundaries, cancellation states, conflicting and non-conflicting slots
Review	Patient.cpp / Doctor.cpp	Remaining validation and duplicate-identity decision outcomes

6. Measurement method and build distinction

Coverage was collected from build-llvm/hms.exe with llvm-cov and the merged profile build-llvm/coverage.profdata. Test-source paths were excluded. The production-only report completed without an LLVM mismatched-data warning. Test targets compile FileManager.cpp with HMS_TESTING conditional code, including different path-validation behaviour. For that reason, this report does not combine coverage mappings from the test executables with the production executable.

7. Limitations

Branch coverage reports execution of instrumented branch outcomes; it does not assess the quality of assertions, prove requirement completeness, or show that every logical input combination is correct. The 228 uncovered branches should be examined against source-level risk and requirements. Some may be defensive or environment-dependent paths; any decision not to test them should be justified.

8. Evidence and reproducibility

Evidence artifact	Purpose
build-llvm/production-coverage-final.txt	Production branch and line summary
build-llvm/production-coverage-final.json	Detailed machine-readable coverage export
build-llvm/ctest-results.xml	JUnit-format record for the 51-test run
build-llvm/coverage.profdata	Merged LLVM instrumentation profile

Execution context: 51 CTest tests passed, 0 failed, in 31.47 seconds. Source revision: 542b90be8adaddf13f7ce900340973eb4c9f5a56.